CS-UY 4563: Machine Learning:

Final Project Written Report

Genre Classification of Spotify Tracks

11:00AM Section

April 29, 2026

Professor Linda N. Sellie

Lionel Hayashi, Jeremy Tob

Introduction

This project explores different machine learning models to predict the song genres based on their features, such as "energy", "key", and "loudness". The dataset was obtained from Kaggle and contains the Spotify track data with 114,000 samples and 21 features. We selected the top 10 most common genres and randomly sampled 200 songs from each genre, creating a balanced dataset of 2,000 total songs. The 10 most common genres are as follows: acoustic, opera, pagode, party, piano, pop, pop-film, power-pop, progressive-house, and punk-rock. We used logistic regression, support vector machines (SVM), and neural networks to classify the songs into ten genre classes. Each model was tested multiple times using different regularization techniques, feature transformations, and normalization methods.

This task is interesting because it provides insight into how models for recommender systems in music and media apps are designed. Music classification is also challenging because audio features often overlap across different genres, making it difficult for models to distinguish between them clearly. This makes the task both relevant and engaging to explore.

Preprocessing

For data preprocessing, we first selected the top most common genres from the original dataset. In the full dataset, genre counts are already evenly distributed at about 1,000 songs per genre, and almost all columns have no missing values; only artists, album_name, and track_name each have 1 missing value. Then, we randomly sampled 200 songs from each genre to create a balanced dataset with 2,000 total songs. We used numeric audio features to train the models. These included popularity, duration, danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, key, mode, and time signature. We found no missing values in the main numeric audio features of the project subsets. Since the genre labels were originally written as text, we converted and mapped them to numbers using LabelEncoder. This allowed the machine learning models to process the genre classes numerically.

The data was split into training, validation, and test set using a 70%/15%/15% ratio. As a result, we ended up with 1,400 songs for the training set, 300 songs for the validation set, and 300 songs for the test set. Stratified samples were used, so that the genre proportions stayed similar across each split. Finally, features were standardized using StandardScaler. For the SVM and logistic regression models, transformed our feature sets using PCA, polynomial features, and log transformation to test whether different feature representations could improve our model performance.

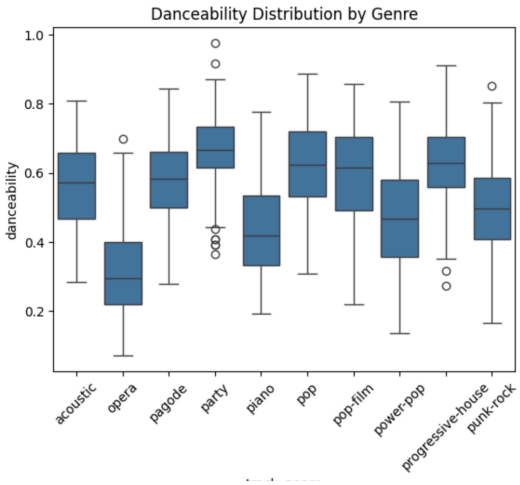

Figure 1 - Danceability Distribution

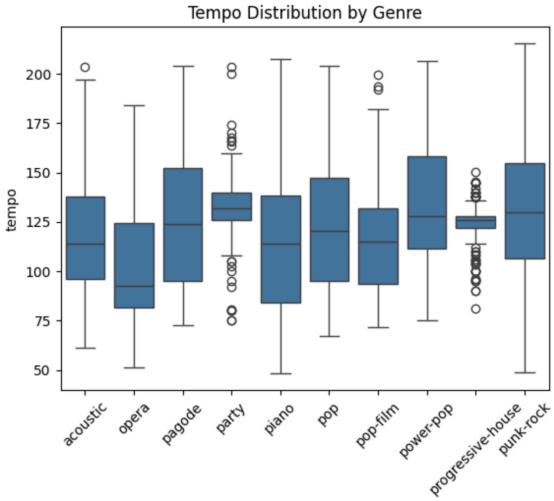

Figure 2 - Tempo Distribution

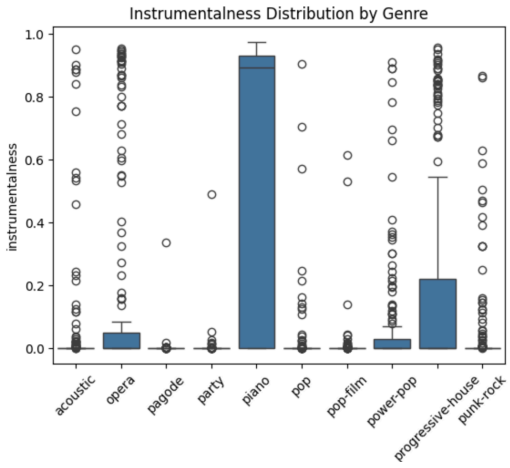

Figure 3 - Instrumentalness Distribution

As observed in the feature distribution plots, certain features such as danceability and tempo exhibit similar distributions across all 10 genres, with heavily overlapping ranges that make it difficult for the model to use them as reliable discriminators. This overlap increases the complexity of the classification task, as these features contribute little signal for distinguishing between genres. In contrast, features like instrumentalness show more distinct distributions across genres. For example, piano tends to score significantly higher than party or pop, making them more informative predictors for the model.

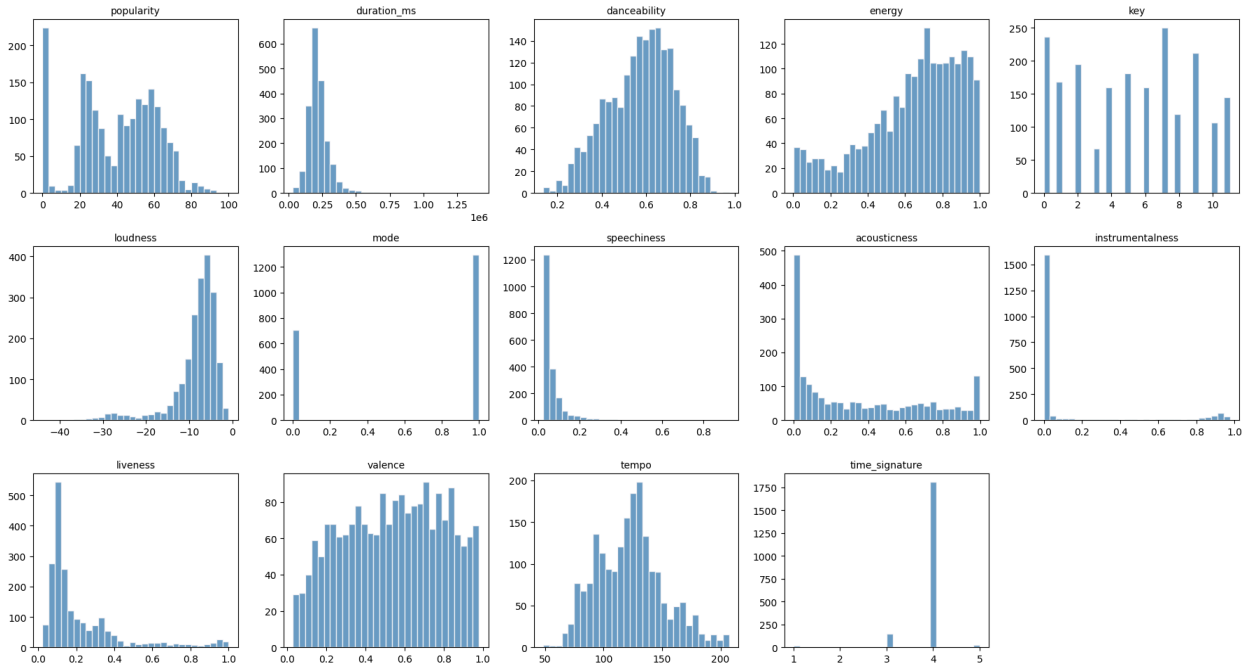

Figure 4 - Feature Distribution of the Dataset

Figure 4 shows the feature distributions across all 14 audio features in our dataset. Several features exhibit heavily right-skewed distributions, such as speechiness, acousticness, instrumentalness, and liveness, where the majority of songs are concentrated near zero with a long tail toward higher values. This skewness motivated our use of a log transformation as one of our feature transformations, as it helps spread out the dense region near zero and gives the model a more balanced view of those features. On the other end, danceability and tempo are the most normally distributed features, centered around 0.6 and 120-130 BPM respectively. Features like mode and time_signature show near-constant values across the dataset meaning these features carry very little help for genre classification. Overall, the distributions confirm that standard scaling alone is insufficient for all features, and that transformation choices need to account for the varying shapes across the feature space.

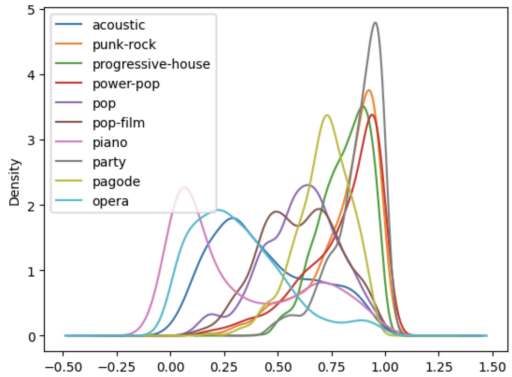

Figure 5 - kernel density estimate for ‘energy’

For all 10 genres, the kernel density estimate for the ‘energy’ feature overlaid on the same axis, showing how much they blur together. The overlapping curves prove that one feature does not cleanly separate genre. At the same time, we can see a peak at 1 for some genres, which shows some common trends between genres. The kernel density graphs highlights the challenge that exists in the nature of our overlapping features between genres.

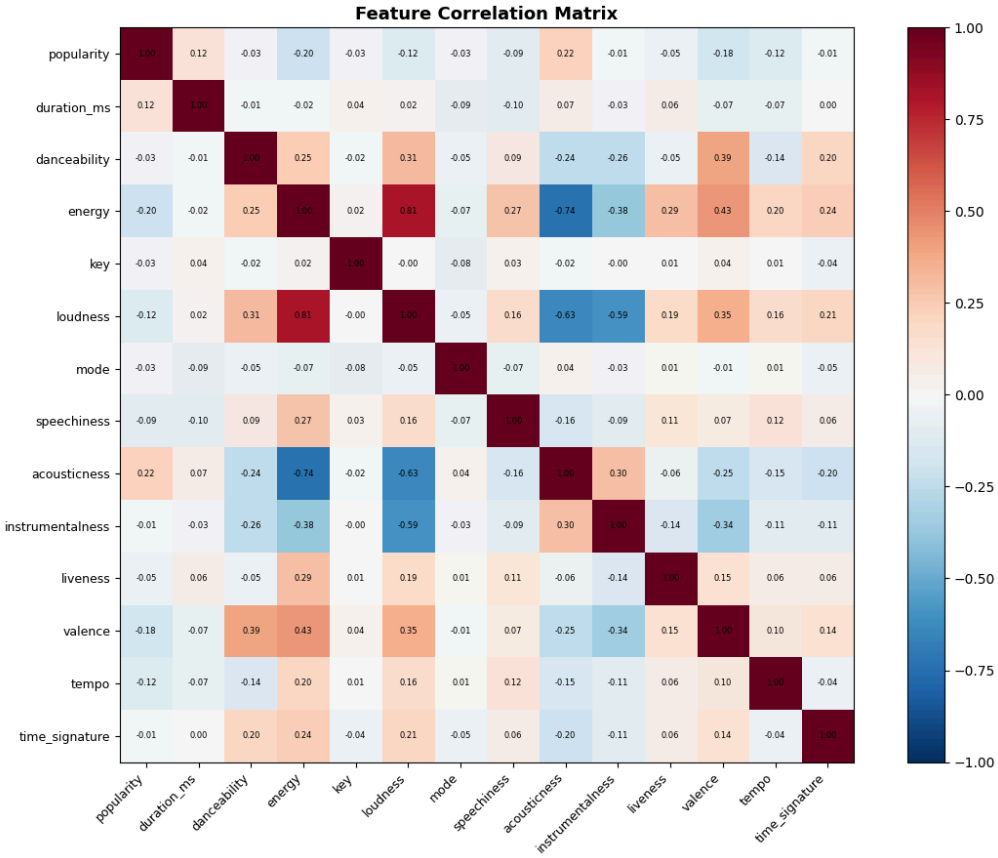

Figure 6 - Feature Correlation Matrix

In the feature correlation matrix, the most notable relationships are between energy, loudness, and acousticness, which are strongly intercorrelated. Energy and loudness have the highest correlation at 0.81, meaning louder songs are almost always more energetic. Both of these features are also strongly negatively correlated with acousticness (-0.74 and -0.63 respectively), which makes intuitive sense, as acoustic tracks tend to be quieter and less intense by nature. Instrumentalness follows a similar pattern, showing a moderate negative correlation with loudness (-0.59). Outside of this cluster, most other feature pairs show weak correlations close to zero, notably key and duration_ms which are nearly independent of everything else. This redundancy among energy, loudness, and acousticness motivated our use of PCA as one of our feature transformations, as PCA is designed to break down correlated features into independent components and potentially give the model a cleaner feature space to work with.

Logistic Regression

First, we used logistic regression to classify genres. Along with the original scaled dataset, we applied three feature transformations: PCA, 2nd degree polynomial, and log-transformed features. Rather than exploring models one at a time, we set up a testbench to systematically evaluate 24 models across different hyperparameter settings. Each model was trained using sklearn's LogisticRegression with the lbfgs solver, 1000 iterations, and a fixed random state of 42. We chose lbfgs because it supports L2 regularization and performs well on small to medium sized datasets like ours. The regularization strength is controlled by the parameter C, which is the inverse of the regularization strength. A small C means strong regularization, and a large C means weak regularization. We swept C across six values: [0.01, 0.1, 1, 10, 50, 100]

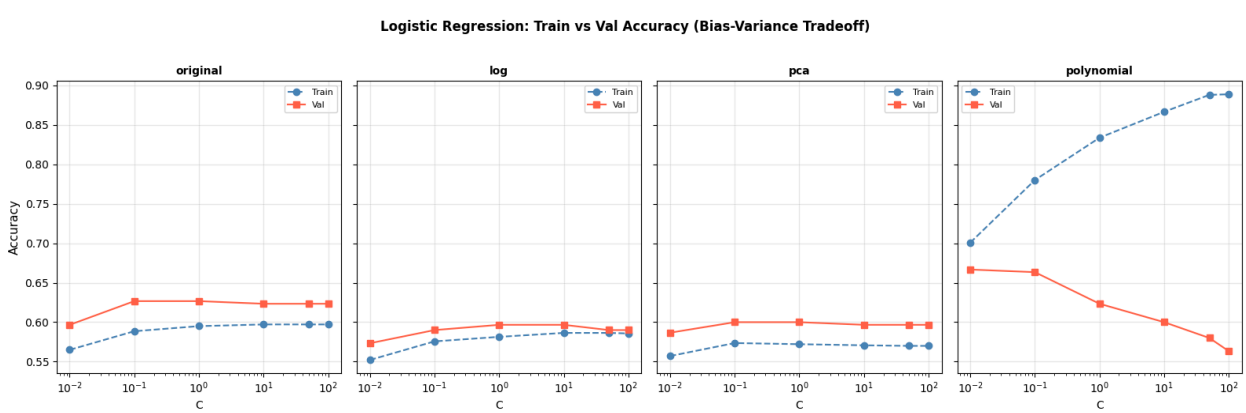

Figure 7 - Train vs Val Accuracy (3 transformations and original

Our first observation was that the polynomial transformation with no regularization (C = 100) achieved the highest training accuracy at 0.88, but its validation accuracy was among the lowest at 0.563. The gap between training and validation accuracy here was the largest across all models, which is a clear sign of overfitting. The model was memorizing the training data rather than learning patterns that generalize. This makes sense given that the polynomial transformation takes our original 14 features to 119 features, giving the model a large number of parameters to fit relative to our 1400 training samples.

This told us that the polynomial transformation had potential, but needed to be restricted with stronger regularization. As we increased regularization by lowering C, the training accuracy dropped but the validation accuracy climbed. By C = 0.01, our strongest regularization setting, the polynomial transformation achieved the best validation accuracy of any model at 0.667, as shown in Figure 7. The train/val gap also narrowed significantly, indicating the regularization was effectively controlling overfitting.

The log and PCA transformations told a different story. The log transformation consistently had the smallest gap between training and validation accuracy across all C values, meaning it was the most stable but also the least expressive. However, it never reached the highs of the polynomial. PCA behaved similarly, sitting between the original and log transformations in most settings. Neither transformation benefited as dramatically from increased regularization as the polynomial did, suggesting they were not overfitting to begin with but also had less room to improve.

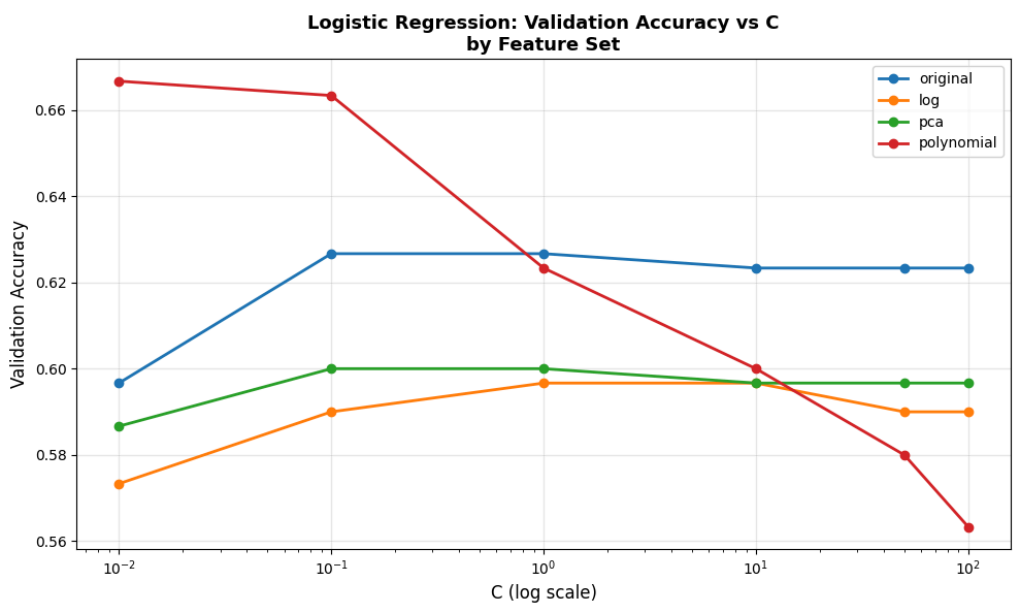

Figure 8 - Validation Accuracy vs. C

As seen in Figure 8, the original scaled features was in the middle, outperforming PCA and log at some regularization values. Although the differences are small, the original dataset wins among those three.

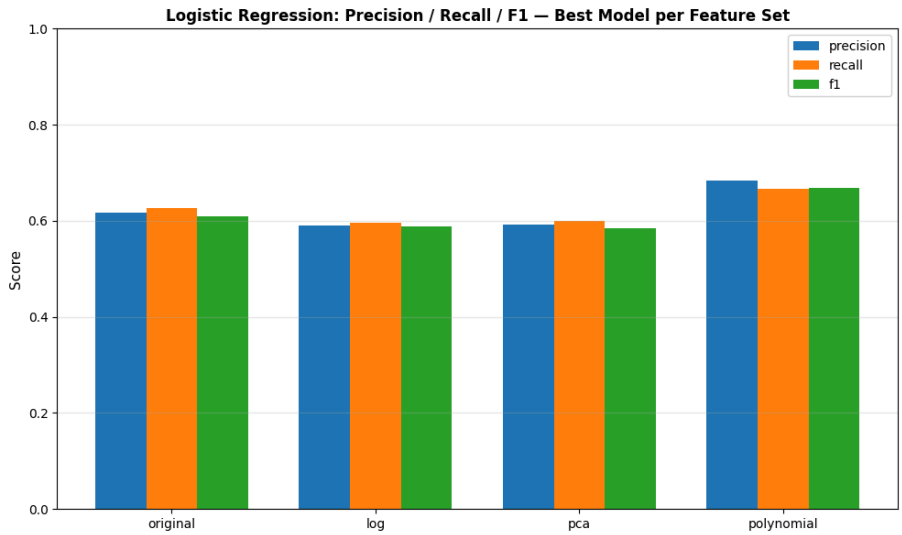

Figure 9 - Logistic Regression Precision, Recall, F1 (3 transformations and original)

Finally, looking at precision, recall, and F1 across all feature sets in Figure 9, the polynomial model with C = 0.01 outperformed all others across every metric. The gap was most visible in F1, where the polynomial led by a meaningful margin over the next best transformation.

Support Vector Machine (SVM)

Next, we decided to model our dataset with SVM. In order to run this model, we imported the SVM model using SVC from sklearn. We used an RBF kernel, which helps the model capture more complex, non-linear relationships between the audio features and the song genres. The following are the four different feature sets we tested on: original scaled features, PCA-transformed features, Polynomial degree of 2 features, Log-transformed features. The original scaled features gave us a baseline using the raw data after standardization. The PCA-transformed features were used to reduce redundancy and noise by compressing the data into fewer components. The polynomial degree-2 features allowed us to capture interaction effects between variables, since genre may depend on combinations of audio features rather than single features alone. The log-transformed features helped reduce skewness in the data and make some feature distributions more balanced.

Then, we tested different C values for L2 regularization: 0.01, 0.1, 1, 10, 50, 100. For each feature set and C value, we trained an SVM model and measured its performance using: training accuracy, validation accuracy, precision, recall, and F1 score.

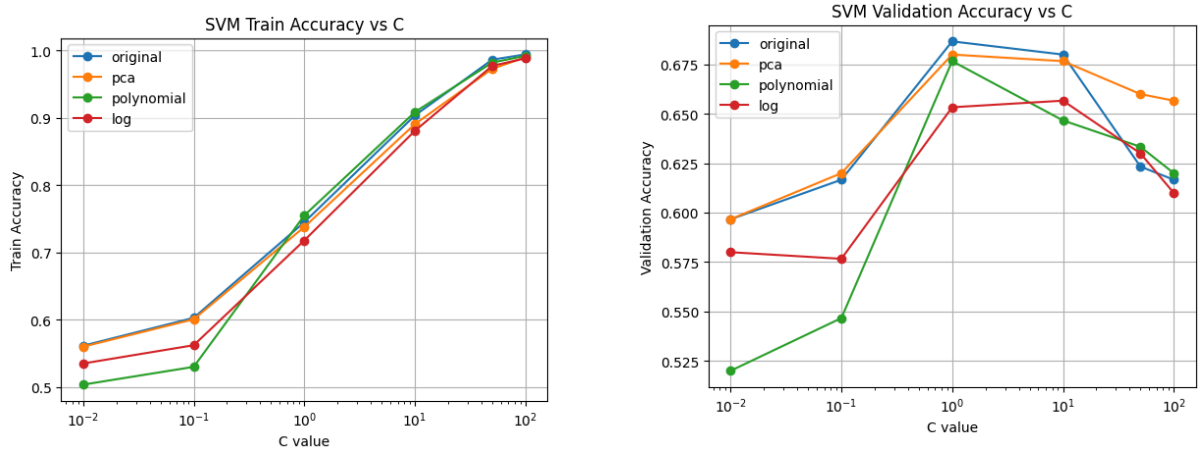

Figure 10a, b SVM: Training and Validation accuracy vs C for each feature set

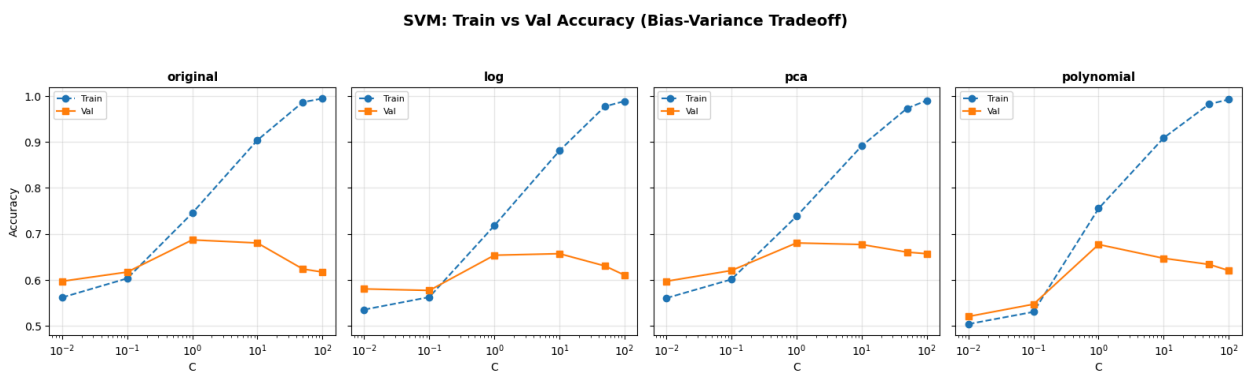

Figure 11 - SVM: Training and Validation accuracy vs C for each feature set in separate graphs

As part of our design process, we initially started with smaller C values and then gradually increased them to observe how model performance changed. This allowed us to identify the point where the model began to overfit, as training accuracy kept rising while validation accuracy stopped improving.

According to Figure 10a, as C values become larger, training accuracy logarithmically approaches 1.0, especially from around C = 10^{-1} onward. There is little difference in training accuracy between C = 10^{-1} and C = 10^{-2}. In contrast, validation accuracy does not improve by the same amount as C increases, as shown in Figure 10b, which suggests that the model begins to overfit at higher C values, especially after C = 1. This suggests the usual bias-variance trade-off: at very low C, the model is too constrained and underfits, leading to high bias and lower train and validation accuracy; at very high C, the model fits the training data too closely, causing overfitting and higher variance, which hurts validation performance.

Overall, we found that moderate C values, especially C = 1, seem to generalize better than very small or very large values, and these patterns are consistent across all four feature sets.

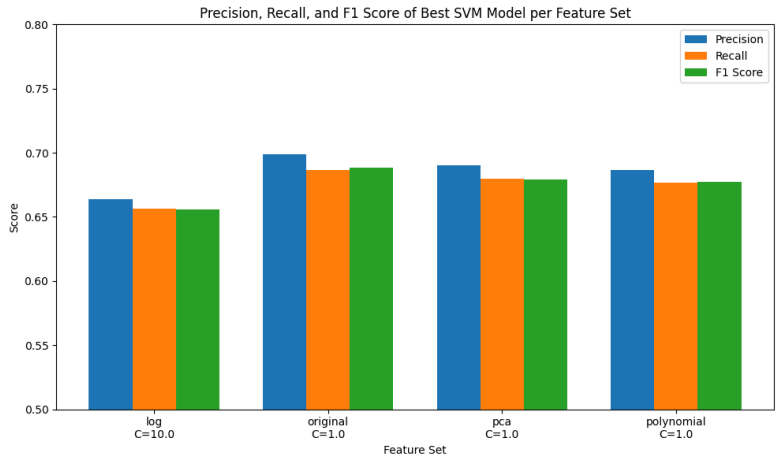

Figure 12 - SVM Precision, Recall, F1 (3 transformations and original)

Looking at Figure 12, the results of precision, recall, and F1 scores are quite similar across the four best feature sets, with only small differences. The original feature set with C = 1 performed the best overall, with the highest precision, recall, and F1 score. PCA and polynomial features with C = 1 followed closely and had very similar results, showing solid but slightly lower performance. I can infer that these extra transformations did not significantly improve the model. This suggests that the original scaled features already contained most of the useful information for classification. In contrast, the log-transformed feature set had the lowest scores across all three metrics, showing that this transformation was less effective for our dataset. Another clear trend is that, within each feature set, precision, recall, and F1 score are very close to one another, which suggests the best SVM models were fairly balanced in their classification performance.

We selected the SVM polynomial model with C = 10 and retrained it to generate a confusion matrix for the results, shown in Figure 13.

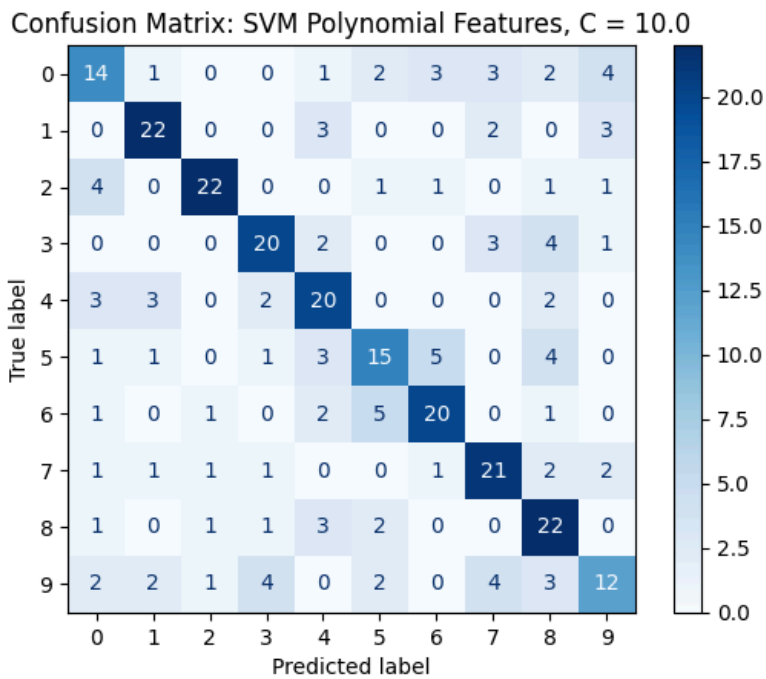

Figure 13 Confusion Matrix: SVM Polynomial (C = 10.0)

Neural Network

Finally, we decided to model our dataset with neural network models. We trained each neural network with a Rectified Linear Unit (ReLu) hidden activations, a softmax output layer, cross-entropy loss, L2 regulation, and Xavier weight initialization. We tested five neural network architectures: Linear has 14 input nodes and 10 output nodes. Shallow has 14 input nodes, one hidden layer with 30 nodes, and 10 output nodes. DeeperNarrow has 14 input nodes, two hidden layers with 20 nodes each, and 10 output nodes. Wider has 14 input nodes, one hidden layer with 64 nodes, and 10 output nodes. DeepWide has 14 input nodes, two hidden layers with 64 and 32 nodes, and 10 output nodes. We chose these five architectures to provide a clear comparison between simple and complex networks and to help identify the best balance between learning capacity and overfitting.

Then, we tested different λ values for L2 regularization: 0, 0.0001, 0.001, 0.01, 0.1, 1.0. For each feature set and C value, we trained all neural network models using the same hyperparameters, with 2,000 iterations and a learning rate of 0.05, to ensure a fair comparison across models. We measured its performance using: training accuracy, validation accuracy, precision, recall, and F1 score.

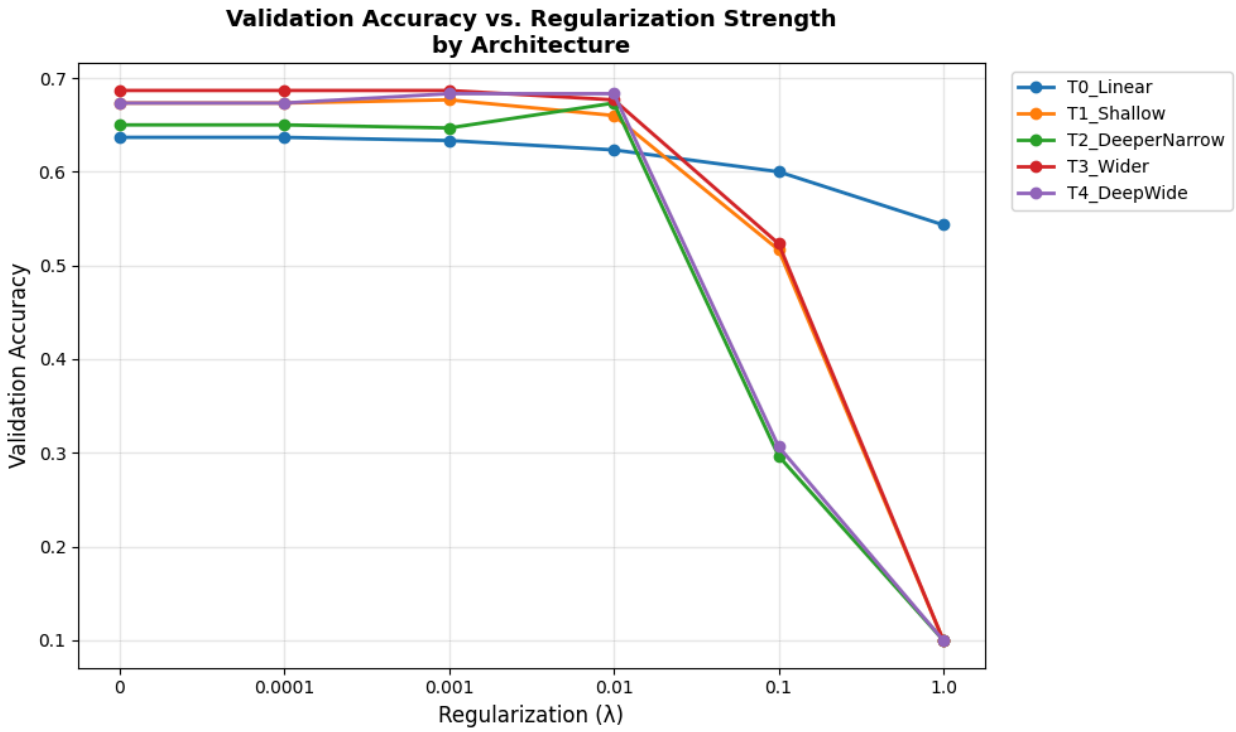

Figure 14 Validation Accuracy vs. Regularization Strength by Architecture

According to Figure 14, validation accuracy stays fairly high and stable when regularization is small, especially from λ = 0 to 0.01. After λ = 0.01, validation accuracy drops sharply for most neural network architectures. This shows that a small amount of regularization can help, but too much regularization causes underfitting and weakens model performance.

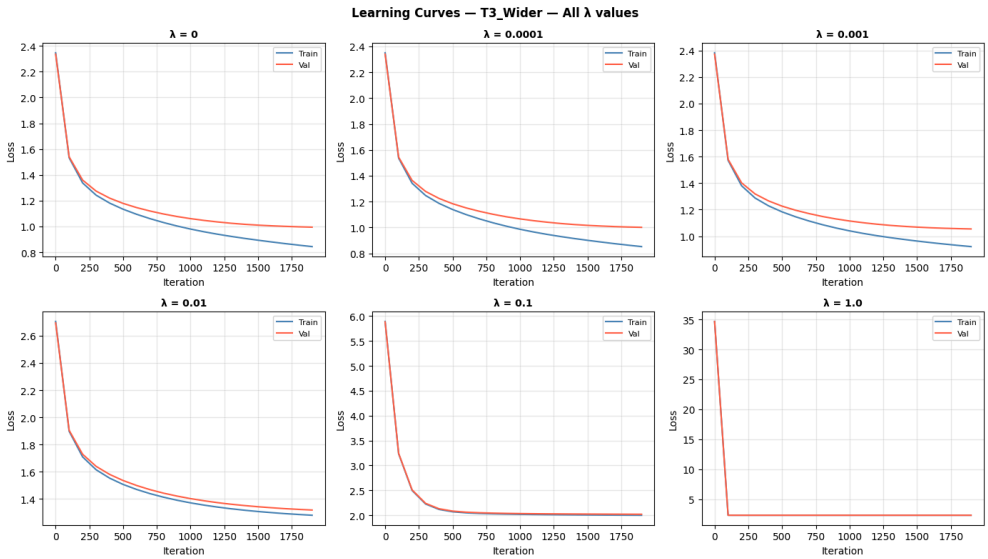

Figure 15 - Learning Curves for DeepWide model

We started with very small values, such as 0, 0.0001, and 0.001, because we wanted the model to learn complex patterns without being restricted too much. These settings gave the best learning curves, with both training and validation loss decreasing to relatively low levels, as shown in Figure 15. At the same time, the gap between training and validation suggests some overfitting, especially when λ is very small. We then increased λ to 0.01, 0.1, and 1.0 to examine what happens when regularization becomes stronger. As λ increased, both training and validation loss stayed higher, showing that the model was being constrained too much and could not learn the data as well. This was most clear at λ = 0.1 and 1.0, where the losses remained very high, indicating underfitting. The results show that small λ values were the best design choice for the DeepWide architecture.

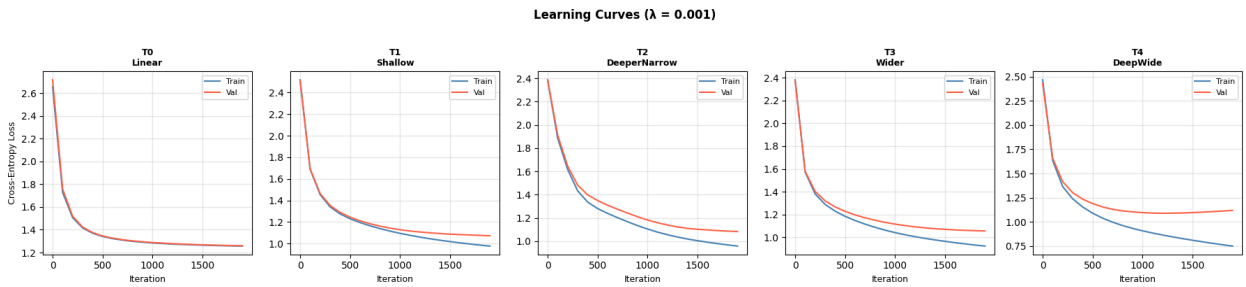

Figure 16 - Learning curve (λ = 0.001) for each architectures

Once we learned that small λ values were the best design choice for the DeepWide architecture, we wanted to see how changing the architectures affected cross-entropy loss with a constant learning curve of λ = 0.001. According to Figure 16, across all five neural network architectures, both training and validation cross-entropy loss decrease quickly at the beginning and then improve more slowly over time. This shows that all models are learning early in training. The deeper and wider models end with lower training loss than the simpler models, meaning they fit the training data better. However, validation loss stays higher than the training loss for almost all models, except Linear model, which suggests some overfitting. Among the architectures, DeepWide achieves the lowest training loss, but its validation loss levels off more and slightly rises near the end, showing the clearest sign of overfitting. Wider architecture achieved the second lowest training loss, but the lowest validation loss.

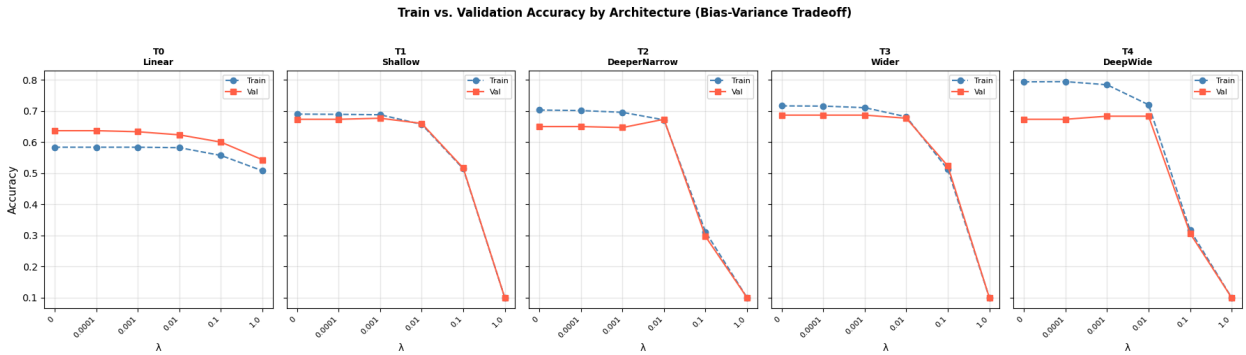

Figure 17 - Train vs. Validation Accuracy by Architecture (Bias-Variance Tradeoff)

Another thing we realized after training the models is that as the architecture becomes larger, they begin with higher training accuracy, but they also show larger gaps from validation accuracy at low λ, indicating greater overfitting risk, as shown in Figure 17. In contrast, the Linear model is more stable but has lower overall accuracy, suggesting higher bias. Shallow, DeeperNarrow, and Wider improve validation accuracy while keeping the gap moderate at small λ values. DeepWide has the highest training accuracy, but it also shows the clearest overfitting at low λ. Additionally, when λ becomes too large, both training and validation accuracy drop sharply for all models, which means too much regularization causes underfitting. Overall, small λ values give the best results, while very large λ hurt performance.

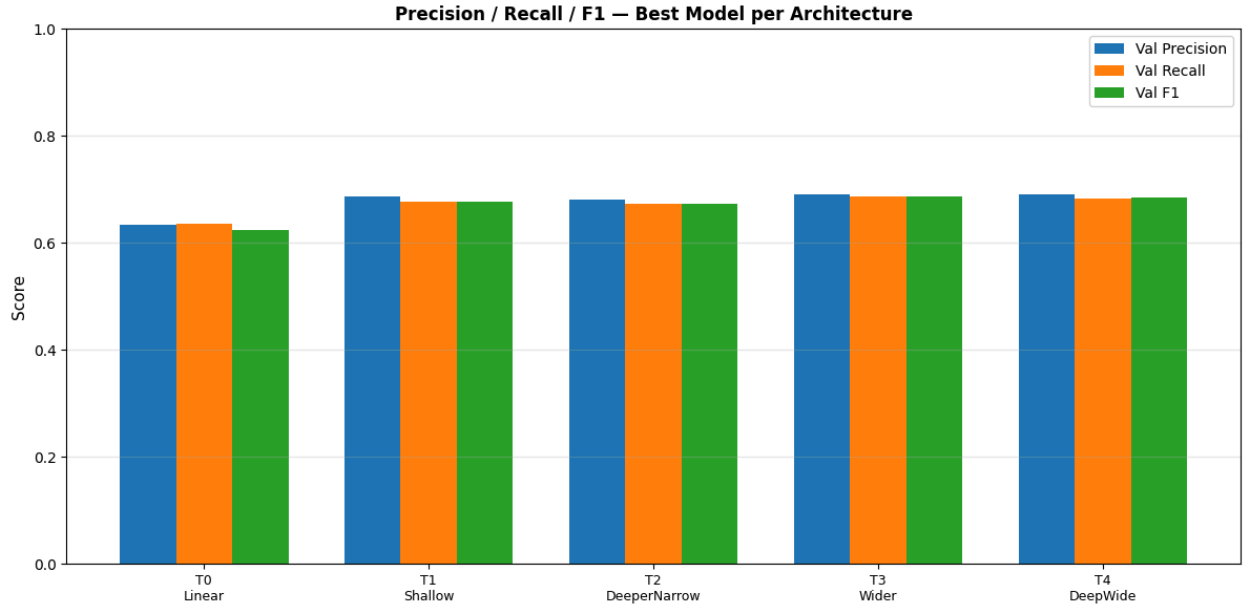

Figure 18 - Neural Network Precision, Recall, F1 (5 architectures)

According to Figure 18, the larger neural network architectures generally performed better than the simpler ones. The Linear model had the lowest precision, recall, and F1 score, while the deeper and wider models achieved the highest scores. Among them, DeepWide performed the best overall, though the differences between Shallow, DeeperNarrow, Wider, and DeepWide were small. This suggests that adding hidden layers and more neurons helped improve performance, but the improvement became smaller as the models got more complex.

Results

Logistic Regression

Feature	Metric	C = 0.01	C = 0.1	C = 1.0	C = 10.0	C = 50.0	C = 100.0
Original	Train Acc	0.565	0.589	0.595	0.597	0.597	0.597
	Val Acc	0.597	0.627	0.627	0.623	0.623	0.623
	Val Precision	0.598	0.616	0.619	0.615	0.615	0.615
	Val Recall	0.597	0.627	0.627	0.623	0.623	0.623
	Val F1	0.581	0.610	0.615	0.611	0.611	0.611
Log	Train Acc	0.552	0.576	0.581	0.586	0.586	0.586
	Val Acc	0.573	0.590	0.597	0.597	0.590	0.590
	Val Precision	0.560	0.582	0.590	0.591	0.586	0.586
	Val Recall	0.573	0.590	0.597	0.597	0.590	0.590
	Val F1	0.557	0.580	0.588	0.589	0.583	0.583
Pca	Train Acc	0.557	0.574	0.572	0.571	0.570	0.570
	Val Acc	0.587	0.600	0.600	0.597	0.597	0.597
	Val Precision	0.582	0.592	0.593	0.593	0.593	0.593
	Val Recall	0.587	0.600	0.600	0.597	0.597	0.597
	Val F1	0.570	0.585	0.586	0.584	0.584	0.584
Polynomial	Train Acc	0.701	0.780	0.834	0.866	0.888	0.889
	Val Acc	0.667	0.663	0.623	0.600	0.580	0.563
	Val Precision	0.684	0.673	0.629	0.607	0.584	0.573
	Val Recall	0.667	0.663	0.623	0.600	0.580	0.563
	Val F1	0.668	0.664	0.623	0.600	0.581	0.567

Feature Set	Best C	Val Accuracy	Test Accuracy	Test Precision
Original	0.1	0.627	0.597	0.594

Log	1	0.597	0.597	0.598
PCA	0.1	0.6	0.6	0.607
Polynomial	0.01	0.667	0.630	0.644

SVM

Feature	Metric	C = 0.01	C = 0.1	C = 1.0	C = 10.0	C = 50.0	C = 100.0
Original	Train Acc	0.561429	0.602857	0.745000	0.903571	0.986429	0.994286
	Val Acc	0.596667	0.616667	0.686667	0.680000	0.623333	0.616667
	Precision	0.612068	0.631913	0.698647	0.685024	0.626856	0.618548
	Recall	0.596667	0.616667	0.686667	0.680000	0.623333	0.616667
	F1 Score	0.594356	0.615959	0.688275	0.679765	0.622574	0.614625
Log	Train Acc	0.560000	0.600714	0.737857	0.890714	0.972857	0.990000
	Val Acc	0.596667	0.620000	0.680000	0.676667	0.660000	0.656667
	Precision	0.611983	0.631572	0.690072	0.685235	0.665181	0.665221
	Recall	0.596667	0.620000	0.680000	0.676667	0.660000	0.656667
	F1 Score	0.594275	0.616963	0.679209	0.676629	0.660461	0.658153
PCA	Train Acc	0.503571	0.530000	0.755000	0.908571	0.982143	0.992143
	Val Acc	0.520000	0.546667	0.676667	0.646667	0.633333	0.620000
	Precision	0.610574	0.617161	0.686242	0.648382	0.637199	0.622605
	Recall	0.520000	0.546667	0.676667	0.646667	0.633333	0.620000
	F1 Score	0.531680	0.556739	0.677067	0.646578	0.632022	0.617855
Polynomial	Train Acc	0.535000	0.562143	0.717857	0.880714	0.977143	0.988571
	Val Acc	0.580000	0.576667	0.653333	0.656667	0.630000	0.610000

	Precision	0.602537	0.593818	0.666536	0.663689	0.636502	0.612643
	Recall	0.580000	0.576667	0.653333	0.656667	0.630000	0.610000
	F1 Score	0.578517	0.575733	0.653554	0.655864	0.628941	0.608649

Feature Set	Best C	Val Accuracy	Test Accuracy	Test Precision
Polynomial	1	0.676667	0.640000	0.649836
Original	1	0.686667	0.656667	0.668317
Log	10	0.656667	0.476667	0.486666
PCA	1	0.680000	0.660000	0.671958

Neural Network

Architecture	Metric	λ = 0	λ = 0.0001	λ = 0.001	λ = 0.01	λ = 0.1	λ = 1.0
Linear	Train Acc	0.5836	0.5836	0.5836	0.5821	0.5571	0.5079
	Val Acc	0.6367	0.6367	0.6333	0.6233	0.6	0.5433
	Val Precision	0.6332	0.6332	0.628	0.615	0.602	0.5698
	Val Recall	0.6367	0.6367	0.6333	0.6233	0.6	0.5433
	Val F1	0.625	0.625	0.6203	0.6089	0.581	0.4918
Shallow	Train Acc	0.69	0.6893	0.6879	0.6571	0.5143	0.1
	Val Acc	0.6733	0.6733	0.6767	0.66	0.5167	0.1
	Val Precision	0.6833	0.6833	0.6866	0.6663	0.4624	0.01
	Val Recall	0.6733	0.6733	0.6767	0.66	0.5167	0.1
	Val F1	0.6741	0.6741	0.6775	0.6592	0.4618	0.0182
DeeperNarrow	Train Acc	0.7029	0.7014	0.6957	0.6714	0.3114	0.1
	Val Acc	0.65	0.65	0.6467	0.6733	0.2967	0.1
	Val Precision	0.6645	0.6645	0.6593	0.6807	0.2324	0.01

	Val Recall	0.65	0.65	0.6467	0.6733	0.2967	0.1
	Val F1	0.6524	0.6524	0.6487	0.6736	0.2457	0.0182
Wider	Train Acc	0.7164	0.7157	0.7107	0.6814	0.5121	0.1
	Val Acc	0.6867	0.6867	0.6867	0.6767	0.5233	0.1
	Val Precision	0.6917	0.6917	0.692	0.684	0.4739	0.01
	Val Recall	0.6867	0.6867	0.6867	0.6767	0.5233	0.1
	Val F1	0.6863	0.6863	0.6863	0.6761	0.4665	0.0182
DeepWide	Train Acc	0.7936	0.7943	0.7843	0.72	0.3171	0.1
	Val Acc	0.6733	0.6733	0.6833	0.6833	0.3067	0.1
	Val Precision	0.6826	0.6826	0.6917	0.6859	0.2367	0.01
	Val Recall	0.6733	0.6733	0.6833	0.6833	0.3067	0.1
	Val F1	0.675	0.6743	0.6847	0.6823	0.2502	0.0182

Feature Set	Best λ	Val Accuracy	Test Accuracy	Test Precision
Linear	0	0.6367	0.6033	0.6
Shallow	0.001	0.677	0.653	0.658
DeeperNarrow	0.01	0.673	0.636	0.644
Wider	0	0.687	0.677	0.682
DeepWide	0.001	0.6833	0.627	0.633

Conclusion

For logistic regression, the main pattern was that the choice of feature transformation mattered more than the regularization strength. As shown in Figure 8, the polynomial transformation stood out clearly from the other three feature sets across almost all C values. The original, log, and PCA features produced similar validation accuracies throughout the sweep, typically ranging between 0.57 and 0.63, with no single transformation pulling ahead consistently. The polynomial transformation, on the other hand, showed a much wider range, starting at its best validation accuracy of 0.667 at C = 0.01 and degrading steadily as C increased, dropping to 0.563 at C = 100. This pattern reflects a classic overfitting story: at low regularization, the model has 119 features to work with and quickly starts memorizing the training data, pushing training accuracy up to 0.88 while validation accuracy falls. Increasing regularization by lowering C forced the model to focus on the most important interactions rather than fitting noise, which is why the strongest regularization setting produced the best generalization. Very small C values slightly hurt the original, log, and PCA models by making them too constrained, but the effect was much smaller since those feature sets have far fewer parameters to begin with. Overall, the best logistic regression result came from the polynomial transformation with C = 0.01, achieving a validation accuracy of 0.667 and a validation F1 of 0.668, which was the highest validation performance across all logistic regression models and outperformed both the log and PCA transformations by roughly 0.07 to 0.09 in accuracy.

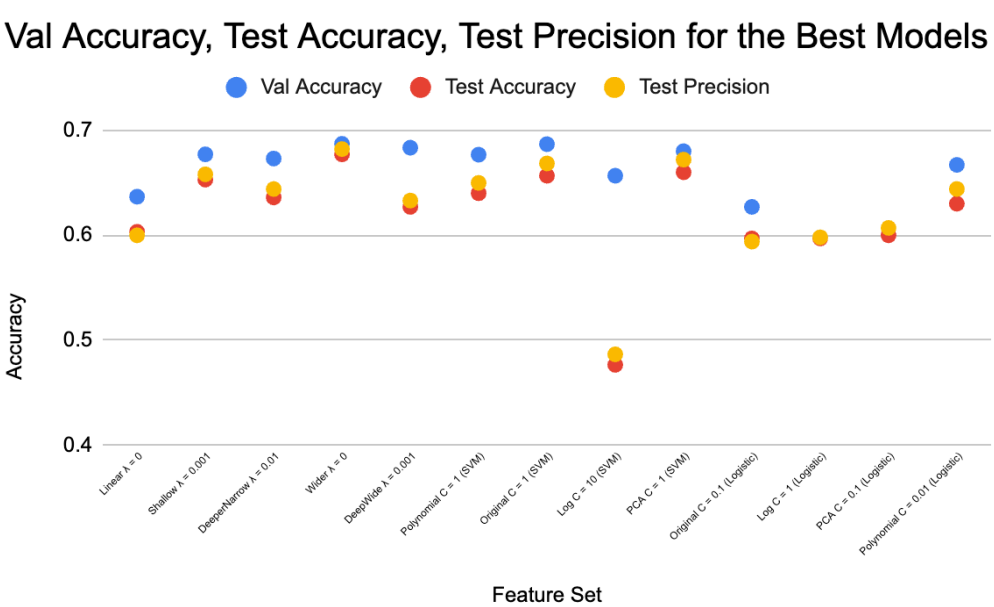

Figure 19 - A Chart of all Key Findings

For SVM, the main pattern was that a moderate C worked best. In Figure #11, training accuracy kept increasing as C became larger, but validation accuracy improved only up to a point and then flatted or dropped, which is a clear sign that very large C caused overfitting. Very small C values made the model too constrained, so they underfit and had higher bias. Among the feature sets, the best SVM result came from the PCA feature set with C = 1, which achieved about 66% test accuracy and 67.2% test precision. This was slightly better than the original features with C = 1, which had about 65.7% test accuracy and 66.8% test precision, even though the original feature set had a 0.6667 higher validation accuracy. The polynomial feature set with C = 1 gave similar, but slightly lower results: 64% test accuracy and 64.9836% test precision. The log-transformed feature set, on the other hand, recorded the lowest test accuracy of 47.7% and 48.7% test precision. Additionally, SVM was robust across several reasonable feature representations. The Original, PCA, and polynomial feature sets all ended up in a very similar performance range, with only small differences in validation precision, recall, and F1 score. That consistency suggests the RBF kernel was already able to capture useful nonlinear structure without relying heavily on manual feature transformation. At the same time, the log-transformed feature set clearly underperformed, with a relatively low test accuracy of 47.7%. Although it was meant to reduce skewness in some features, it may have also removed useful information that the SVM needed to better distinguish between genres.

For neural networks, the best model was Wider with a λ value of 0, which achieved a validation accuracy of 68.67%, test accuracy of 67.7%, test recall of 68.2%, test F1 score of 67.4%. A key takeaway is that adding some complexity helped, but only up to a point. In general, the larger architectures performed better than the simpler ones. However, the improvement became smaller as models became more complex. This suggests that increasing model size gave diminishing returns on this dataset rather than a major jump in performance. As shown in Figure 17, as the networks became deeper and wider, training accuracy increased, but the gap between the training and validation also became larger, which means overfitting became more likely. At the same time, when λ became too large, both training and validation accuracy dropped sharply, so the model underfit because regularization was too strong. This is especially clear at $\lambda = 1.0$; all models except Linear had the same exact training accuracy, val accuracy, val precision, val recall, and val F1. It suggests that when $\lambda = 1.0$, the regularization was so strong that most of the neural networks were forced into almost the same simple behavior, no matter how deep or wide they were. In other words, the larger models could not use their extra layers and neurons effectively because the L2 penalty pushed the weights to stay very small. As a result, Shallow, DeeperNarrow, Wider, and DeepWide all ended up making very similar predictions, which is why they produced similar training accuracy, validation accuracy, precision, recall, and F1 score. Still, it is interesting to note the surprising

performance of our linear model with a 0.6 test accuracy, which suggests that our data is more linearly separable than we previously thought.

For each model type, there are a few directions worth exploring to improve performance of the model. In logistic regression, experimenting with L1 regularization instead of L2 could act as automatic feature selection across the 119 polynomial features, potentially improving generalization. Additionally, we could extend the polynomial transformation to degree 3 to explore three way interaction between features (e.g. energy, loudness, and acousticness together) which might reveal genre structure that pairs alone missed. A useful next step for SVM would be to tune the gamma, since the kernel behavior may change a lot depending on that setting. For neural networks, it would make sense to try dropout, early stopping, or better learning-rate tuning, since those could reduce overfitting without forcing the model to underfit. Additionally, running each neural network multiple times with different random initial weights would make the results more reliable, since performance can vary slightly depending on the starting weights. Additionally, we could test different sets of genres to see whether the specific genre choices affect model performance.

In general, all of our models could have benefited from additional data in the training set, and each model for a slightly different reason. Logistic regression had the tightest feature-to-sample ratio problem, especially with the polynomial transformation. 1400 samples across 119 features is thin, and more data would let heavier regularization be relaxed without overfitting. Neural networks suffered the most visibly from data scarcity. The deeper models clearly had the capacity to learn more complex patterns but couldn't generalize because 1400 samples was not enough to train many weights reliably. With more data, we would have seen a wider gap between shallow and deep neural networks. SVM could have used more data to support vectors spread across the feature space to draw good boundaries. With 140 samples per genre, some boundaries were likely not well defined. Finally, all three model types converged to a roughly similar validation accuracy ceiling of 62-65% despite being different in complexity. That is a strong indicator that the bottleneck is the data itself, both the quantity and the ambiguity in audio features as genre predictors, rather than any modeling choice. More data would raise the ceiling for all three, but the genre overlap problem in features would still set an upper limit. Moving forward, richer features such as raw audio spectrograms or lyric-based embeddings, combined with larger and more diverse training sets, would likely be the most impactful next step toward building a more robust genre classifier.

Works Cited

Deeplearningbook.Org, 2026, https://www.deeplearningbook.org/contents/ml.html. Accessed 29 Apr. 2026.

Kaggle. "🎹 Spotify Tracks Dataset." Baselight, 20 Mar. 2025, https://baselight.app/u/kaggle/dataset/maharshipandya_spotify_tracks_dataset. Accessed 28 Apr. 2026.

Ng, Andrew, and Tengyu Ma. CS229 Lecture Notes. 2023, https://cs229.stanford.edu/main_notes.pdf.

"Scikit-Learn: Machine Learning in Python — Scikit-Learn 1.8.0 Documentation." Scikit-Learn.Org, 2025, https://scikit-learn.org/stable/. Accessed 30 Apr. 2026.

"Understanding the Bias-Variance Tradeoff." Fortmann-Roe.Com, 2021, https://scott.fortmann-roe.com/docs/BiasVariance.html. Accessed 29 Apr. 2026.

"W3Schools.Com." W3schools.Com, 2026, https://www.w3schools.com/python/matplotlib_pyplot.asp. Accessed 30 Apr. 2026.

Github Repo Link